

JAS OS

Technical Report

Freestanding 32-bit x86 operating-system implementation

Student	Jubayed Ahmed Sahel
Student ID	2221173642
Course	CSE 323, Section 3
Project version	JAS OS v1.5.4
Platform	Freestanding i686 kernel booted from an ISO in Oracle VirtualBox

github.com/Jubayed-Sahel/JAS-OS

Report structure: system overview · implemented features · demonstrations · STAR challenges · verification · submission links

1. Executive Summary

JAS OS is a bootable, freestanding 32-bit x86 course-project operating system. It boots from an ISO in Oracle VirtualBox and provides a graphical desktop, terminal, applications, and observable kernel subsystems. Live output includes task states, scheduling activity, synchronization values, memory statistics, deadlock safety results, paging state, file-system metadata, storage calculations, and I/O counters.

2. System Overview and Scope

JAS OS transfers the useful operating-system components of an earlier ESP32-S3 mini-kernel to a PC-style x86 environment. The result includes a custom boot path, protected-mode kernel, graphical desktop, terminal, Files, Notes, Task Manager, Calculator, Settings, Clock, and OS Lab.

Included. A BIOS boot path, protected-mode kernel, graphical desktop, terminal, Files, Notes, Task Manager, Calculator, Settings, Clock, OS Lab, and observable implementations for scheduling, synchronization, deadlocks, memory, paging, file systems, storage, and I/O.

Excluded. Networking, ring-3 isolation, production device drivers, and claims of production persistence or security.

3. System Architecture

The BIOS loads a custom El Torito boot image. A 16-bit loader reads the kernel, requests a VESA framebuffer, enables A20, installs the GDT, and enters 32-bit protected mode. The kernel initializes IDT/PIC/PIT, serial, PS/2 input, graphics, tasks, scheduling, memory, paging, synchronization, Banker's algorithm, MiniFS, storage/I/O models, event logging, and the GUI.

Boot and hardware	BIOS/El Torito, VESA, A20, GDT, IDT, PIC, PIT, PS/2
Core execution	Task control blocks, cooperative scheduler, semaphores, event log
Memory and safety	First-fit heap, paging model, FIFO replacement, Banker's algorithm
Storage and I/O	MiniFS, disk scheduling, RAID model, polling, interrupts, DMA
Evidence interface	Desktop, Terminal, Feature Guide, OS Lab, global STOP

4. Implemented Features

Feature	Evidence in JAS OS
OS organization	Live subsystem dashboard
Services and boot	Boot trace and syscall aliases
Processes and IPC	Task states and mailbox send/receive
Threads	Shared-counter workers
CPU scheduling	RR, Priority, FCFS, SJF, timeline
Synchronization	Semaphores and readers-writers
Deadlock avoidance	Banker's safety algorithm
Main memory	First-fit heap and coalescing
Virtual memory	Paging, faults, frames, FIFO
File system	MiniFS directories, files, and allocation comparison
Mass storage	Disk algorithms and RAID model
I/O systems	Polling, interrupts, and DMA counters

5. Demonstration Interface

The terminal Feature Guide and Modules screen map each subsystem to direct commands. Scheduling changes the CPU timeline; readers-writers changes semaphore-protected state; Banker's algorithm calculates a safe sequence; and paging changes frame ownership and fault counts. OS Lab invokes the same kernel commands through buttons. The global stop ends all non-shell activity while keeping the terminal usable.

6. Challenges Using the STAR Format

Challenge 1 — Moving from ESP32-S3 to a Bootable x86 ISO

SITUATION	The previous mini-kernel depended on an ESP32-S3 runtime, while this project needed to boot as a PC operating system in Oracle VirtualBox.
TASK	Preserve the relevant course concepts without a host operating system or standard library and provide keyboard, mouse, and graphical output.
ACTION	I implemented a BIOS/El Torito loader, VESA framebuffer setup, A20, GDT and protected-mode transition, then added IDT/PIC/PIT, serial, PS/2 input, graphics, and freestanding runtime modules.
RESULT	JAS OS boots from its own ISO into an interactive 1024×768 desktop, and the cross-compiler build completes without warnings.

Challenge 2 — Producing Live Evidence Instead of Static Labels

SITUATION	Static interface labels would not show whether the algorithms were connected to changing system state.
TASK	Connect every major subsystem to stateful code and observable results.
ACTION	I separated scheduling, tasks, synchronization, memory, paging, Banker's algorithm, files, storage, and I/O into modules and exposed their changing state in the terminal.
RESULT	Each feature has a terminal command and OS Lab button backed by live kernel state, making state changes and algorithm results directly observable.

6. Challenges Using the STAR Format — Continued

Challenge 3 — Organizing Evidence During Assessment

SITUATION	The command set became difficult to remember, and background output could hide the result I needed to discuss.
TASK	Make my implementation evidence easy to find without replacing the algorithms with static interface text.
ACTION	I added the Feature Guide, Modules screen, readable terminal layout, scrolling controls, and OS Lab shortcuts. Each path opens a direct command for a specific kernel subsystem.
RESULT	The interface now provides a consistent route from a feature name to its source-backed live result.

Challenge 4 — Stopping Every Background Task Safely

SITUATION	Producer-consumer, thread, readers-writers, and counter jobs can generate asynchronous terminal output. The old stop action controlled only one demonstration.
TASK	Stop all background activity without terminating the shell.
ACTION	The global stop handler calls each lab's shutdown, scans the task table, terminates remaining non-shell tasks, and restores scheduler state.
RESULT	stop, stop all, lab stop, and the red STOP button end terminal background activity while the shell remains ready.

7. Verification and Result

Check	Verified result
Kernel	89,280 bytes; build completed with no compiler warnings
ISO	1,523,712 bytes
ISO SHA-256	87AC4F6F57DE50F15DEECB5AE510F4508AA9597E98443D40B7AB85FBBAF271AE
Virtual machine	32-bit BIOS, 1024×768 framebuffer, PS/2 input
Feature coverage	Scheduling, synchronization, memory, paging, files, storage, and I/O
Global stop	Ends non-shell activity and leaves the terminal responsive
Demonstration	4:20 narrated 1080p MP4 using genuine JAS OS captures

8. Submission Links

GitHub repository: <https://github.com/Jubayed-Sahel/JAS-OS>

Bootable ISO: <https://github.com/Jubayed-Sahel/JAS-OS/blob/main/jas-os.iso>

Narrated demonstration: <https://github.com/Jubayed-Sahel/JAS-OS/blob/main/docs/JAS-OS-demo.mp4>

9. Limitations and Scope Boundary

JAS OS is a course-project kernel, not a production operating system. Tasks are cooperative, applications execute in kernel space, and MiniFS is RAM-backed. Paging, disk, RAID, and I/O are transparent software models. The project does not claim ring-3 isolation, production device drivers, networking, persistence, or production security.

Conclusion

The completed JAS OS package combines a bootable x86 kernel, graphical applications, direct terminal commands, live subsystem state, a reproducible build, and a verified VirtualBox ISO. The report, repository, and demonstration materials describe the implemented features consistently.